Nat Curtis

ME 3313

9/10/2025

ME 3313 Honors Project

Introduction

For this project, I am analyzing the linkages of a skid-steer bucket loader, shown in Figure 1. I will explore each of the topics we explore in ME 3313, starting with mobility, and going on to topics including range of motion, position, velocity, acceleration, and forces, and use each of these topics to analyze the linkage. I will then explore synthesis and try to create my own linkage for the bucket loader.

Figure 1: Side view of a Skid

Mobility

Mobility is a measure of how many degrees of freedom a linkage has, and it is calculated with Equation (1), called the Grubler-Kutzbach Mobility equation, where M is the mobility, L is the number of links, J1 is the number of full joints, and J2 is the number of half joints.

$$M = 3(L - 1) - 2J_1 - J_2 \tag{1}$$

A link is a single rigid body. A full joint is a joint with 1 degree of freedom, like a pin or two flat surfaces sliding along each other. A half joint is a joint with 2 degrees of freedom, like a pin in a joint, or two curved surface rolling or slipping along each other. On a kinematic diagram of a linkage, each link is marked with a unique plain number from 1 to the number of links, each full joint is marked with a unique circled number from 1 to the number of full joints, and each half joint is marked with a unique boxed number from 1 to the

number of half joints. As shown in Figure 2, the main linkage of the skid-steer has 6 links, 7 full joints, and 0 half joints. Using the mobility equation as shown in Equation (2), the mobility is calculated to be 1.

$$M = 3(L-1) - 2J_1 - J_2 = 3(6-1) - 2*7 - 0 = 1 \tag{2}$$

Because the mobility is 1, only one input is needed to control the linkage. In this case, the input is a hydraulic piston, which is a combination of links 4 and 6. Figure 3 shows the annotated linkage diagram overlain on the original image of the skid-steer.

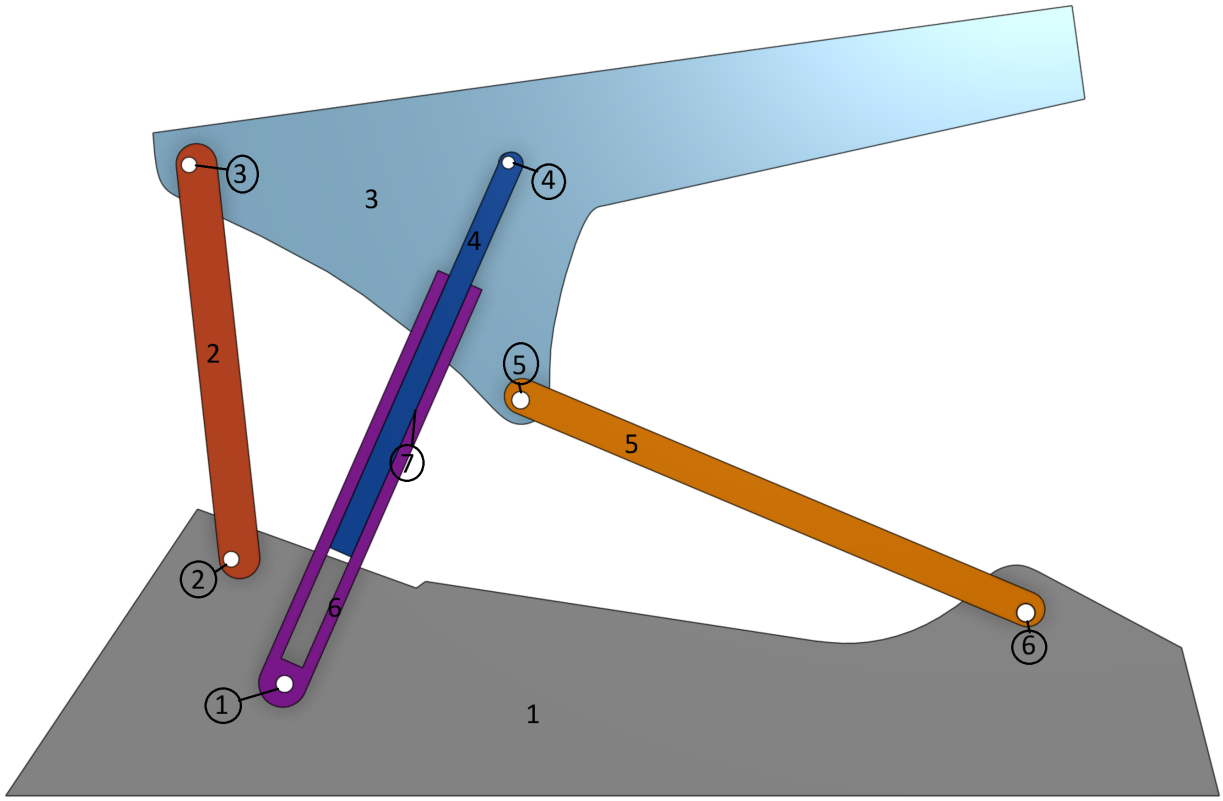

Figure 2: Annotated Linkage

Figure 3: Annotated Linkage Overlain On Skid-Steer

Multiloop Closure Equations

Mobility allows us to determine the degrees of freedom of a linkage, but usually we also need to determine the positions and angles of the individual links. To do this, we can use a method called Multiloop closure. It consists of drawing vectors between joints of each link, then writing equation summing loops of vectors, because the RHS of the equation has to equal zero. This is because the last vector ends where the first vector starts. These vector equations can then be turned into their scalar forms, which can be combined with equations relating rigid geometry of the links, providing a system of equations to solve. When done correctly, the number of unknowns minus the number of scalar and rigid geometry equations should equal the mobility. For the examined linkage, the vectors and loops are labeled as shown in Figure 4. The vector equations are shown below as eqs. (3) and (4).

$$\vec{R_{BA}} + \vec{R_{DB}} - \vec{R_{CA}} - \vec{R_{GC}} - \vec{R_{DG}} = 0 \tag{3}$$

$$\vec{R_{GC}} + \vec{R_{DG}} - \vec{R_{DE}} - \vec{R_{EF}} - \vec{R_{FC}} = 0 \tag{4}$$

These equations can be expanded into their scalar forms as shown in eqs. (5) to (10) below, where each θ is measured counter-clockwise from the horizontal to the tail of the corresponding vector.

$$R_{BA}\cos\theta_{BA} + R_{DB}\cos\theta_{DB} - R_{CA}\cos\theta_{CA} - R_{GC}\cos\theta_{GC} - R_{DG}\cos\theta_{DG} = 0 \tag{5}$$

$$R_{BA}\sin\theta_{BA} + R_{DB}\sin\theta_{DB} - R_{CA}\sin\theta_{CA} - R_{GC}\sin\theta_{GC} - R_{DG}\sin\theta_{DG} = 0 \tag{6}$$

$$R_{GC}\cos\theta_{GC} + R_{DG}\cos\theta_{DG} - R_{DE}\cos\theta_{DE} - R_{EF}\cos\theta_{EF} - R_{FC}\cos\theta_{FC} = 0 \tag{7}$$

$$R_{GC}\sin\theta_{GC} + R_{DG}\sin\theta_{DG} - R_{DE}\sin\theta_{DE} - R_{EF}\sin\theta_{EF} - R_{FC}\sin\theta_{FC} = 0 \tag{8}$$

$$\theta_{GC} - \theta_{DG} = 0 \tag{9}$$

$$\theta_{DB} + \beta_D - \theta_{DE} = 0 \tag{10}$$

In these equations, R_{BA}, R_{DB}, R_{CA}, θ_{CA}, R_{GC}, R_{DE}, R_{EF}, R_{FC}, θ_{FC}, and β_D are known from the geometry of the linkage and do not change. This leaves θ_{BA}, θ_{DB}, θ_{GC}, R_{DG}, θ_{DG}, θ_{DE}, and θ_{EF} as unknowns for a total of 7 unknowns. Taking 7 unknowns minus 6 equations yields 1 degree of freedom, which agrees with the earlier mobility analysis. Also, R_{DG} will be the input because it is controlled by a hydraulic piston.

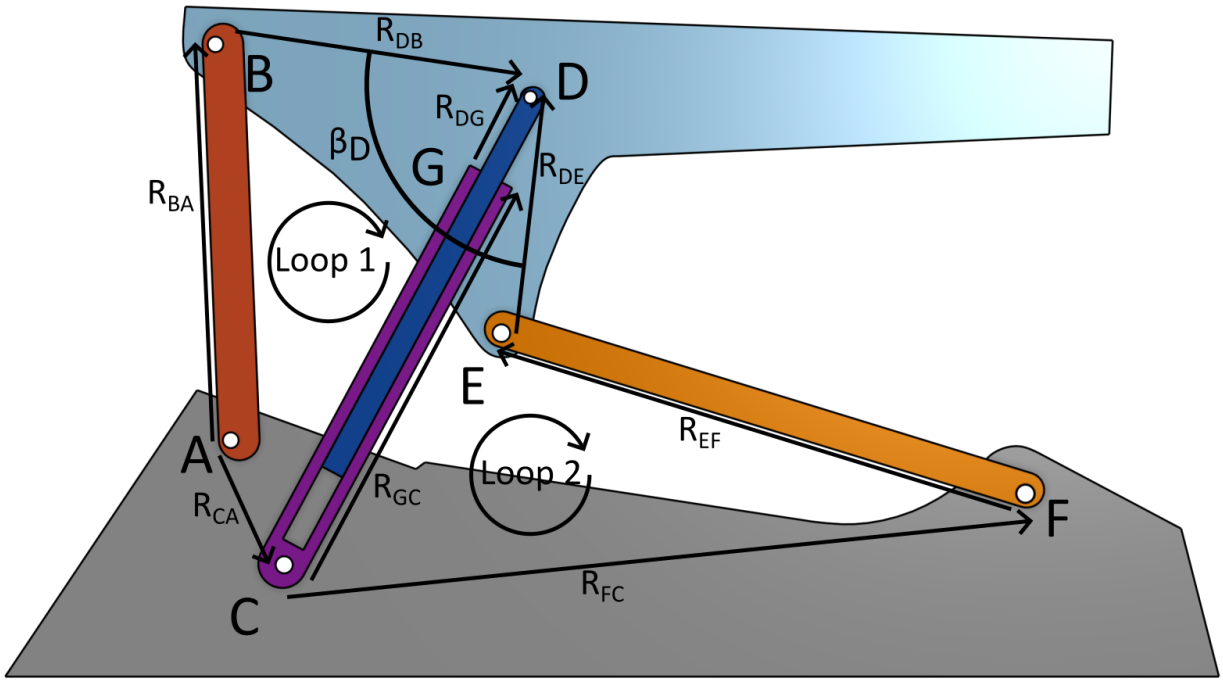

Figure 4: Diagram showing vector for Multiloop Equations

Loop Equation Derivatives

The next step in analyzing the linkage is to take the first and second derivatives of the scalar loop equations, which we can then use to find equations for velocity and acceleration of the links. The first derivatives of eqs. (5) to (10) are as shown below as eqs. (11) to (16), respectively.

$$-R_{BA}\omega_{BA}\sin\theta_{BA} - R_{DB}\omega_{DB}\sin\theta_{DB} + R_{GC}\omega_{GC}\sin\theta_{GC} - \dot{R}_{DG}\cos\theta_{DG} + R_{DG}\omega_{DG}\sin\theta_{DG} = 0 \tag{11}$$

$$R_{BA}\omega_{BA}\cos\theta_{BA} + R_{DB}\omega_{DB}\cos\theta_{DB} - R_{GC}\omega_{GC}\cos\theta_{GC} - \dot{R}_{DG}\sin\theta_{DG} - R_{DG}\omega_{DG}\cos\theta_{DG} = 0 \tag{12}$$

$$-R_{GC}\omega_{GC}\sin\theta_{GC} + \dot{R}_{DG}\cos\theta_{DG} - R_{DG}\omega_{DG}\sin\theta_{DG} + R_{DE}\omega_{DE}\sin\theta_{DE} + R_{EF}\omega_{EF}\sin\theta_{EF} = 0 \quad (13)$$

$$R_{GC}\omega_{GC}\cos\theta_{GC} + \dot{R}_{DG}\sin\theta_{DG} + R_{DG}\omega_{DG}\cos\theta_{DG} - R_{DE}\omega_{DE}\cos\theta_{DE} - R_{EF}\omega_{EF}\cos\theta_{EF} = 0 \quad (14)$$

$$\omega_{GC} - \omega_{DG} = 0 \quad (15)$$

$$\omega_{DB} - \omega_{DE} = 0 \quad (16)$$

The second derivatives of eqs. (5) to (10) are shown below as eqs. (17) to (22), respectively.

$$\begin{aligned} &-R_{BA}\alpha_{BA}\sin\theta_{BA} - R_{BA}\omega_{BA}^2\cos\theta_{BA} - R_{DB}\alpha_{DB}\sin\theta_{DB} \\ &- R_{DB}\omega_{DB}^2\cos\theta_{DB} + R_{GC}\alpha_{GC}\sin\theta_{GC} + R_{GC}\omega_{GC}^2\cos\theta_{GC} - \ddot{R}_{DG}\cos\theta_{DG} \\ &+ 2\dot{R}_{DG}\omega_{DG}\sin\theta_{DG} + R_{DG}\alpha_{DG}\sin\theta_{DG} + R_{DG}\omega_{DG}^2\cos\theta_{DG} = 0 \end{aligned} \quad (17)$$

$$\begin{aligned} &R_{BA}\alpha_{BA}\cos\theta_{BA} - R_{BA}\omega_{BA}^2\sin\theta_{BA} + R_{DB}\alpha_{DB}\cos\theta_{DB} - R_{DB}\omega_{DB}^2\sin\theta_{DB} \\ &- R_{GC}\alpha_{GC}\cos\theta_{GC} + R_{GC}\omega_{GC}^2\sin\theta_{GC} - \ddot{R}_{DG}\sin\theta_{DG} \\ &- 2\dot{R}_{DG}\omega_{DG}\cos\theta_{DG} - R_{DG}\alpha_{DG}\cos\theta_{DG} + R_{DG}\omega_{DG}^2\sin\theta_{DG} = 0 \end{aligned} \quad (18)$$

$$\begin{aligned} &-R_{GC}\alpha_{GC}\sin\theta_{GC} - R_{GC}\omega_{GC}^2\cos\theta_{GC} + \ddot{R}_{DG}\cos\theta_{DG} - 2\dot{R}_{DG}\omega_{DG}\sin\theta_{DG} \\ &- R_{DG}\alpha_{DG}\sin\theta_{DG} - R_{DG}\omega_{DG}^2\cos\theta_{DG} + R_{DE}\alpha_{DE}\sin\theta_{DE} \\ &+ R_{DE}\omega_{DE}^2\cos\theta_{DE} + R_{EF}\alpha_{EF}\sin\theta_{EF} + R_{EF}\omega_{EF}^2\cos\theta_{EF} = 0 \end{aligned} \quad (19)$$

$$\begin{aligned} &R_{GC}\alpha_{GC}\cos\theta_{GC} - R_{GC}\omega_{GC}^2\sin\theta_{GC} + \ddot{R}_{DG}\sin\theta_{DG} + 2\dot{R}_{DG}\omega_{DG}\cos\theta_{DG} \\ &+ R_{DG}\alpha_{DG}\cos\theta_{DG} - R_{DG}\omega_{DG}^2\sin\theta_{DG} - R_{DE}\alpha_{DE}\cos\theta_{DE} \\ &+ R_{DE}\omega_{DE}^2\sin\theta_{DE} - R_{EF}\alpha_{EF}\cos\theta_{EF} + R_{EF}\omega_{EF}^2\sin\theta_{EF} = 0 \end{aligned} \quad (20)$$

$$\alpha_{GC} - \alpha_{DG} = 0 \quad (21)$$

$$\alpha_{DB} - \alpha_{DE} = 0 \quad (22)$$

Code Introduction

To solve these equations, I used Python code taking advantage of the fsolve function from the SciPy library. This is a numerical solver function that takes in a set of equations and initial guesses and outputs a solution. Depending on the initial guess, this solution can be incorrect, or a valid solution but not the correct solution for the actual problem constraints. To increase the chancess significantly of a valid solution being generated, I generated a range of initial condition options for each unknown variable and then generated a solution for every combination of initial conditions. Taking these, I kept only ones that had little error when plugged back into the multiloop equations and each variable was within an estimated range of motion. The known link lengths that were plugged into the equations are shown below, and the input length is shown under that. These measurements are only estimates, however, because I found them by finding the length of the loader in its documentation, then measured the length in a side view image of the loader. I took the ratio of these, then measured each link length and and angle in the picture and multiplied each link length by the ratio to get the estimated actual link length. Angles however are not affected by scaling an image up or down though, so I used the measured angles as they were.

I then took this solution and used it to find solutions along a range of input angles. Because angles that are close together will have similar solutions, I was able to start at my calculated solution and use it as the initial guess for a slightly different angle. This would quickly converge to another solution close by, yielding a correct solution. Using this new solution as my next guess, I was able to repeat this and get the solution for a range of input angles. I then wrote equations for the absolute position of each joint in the linkage, with the origin at Joint C. The vector forms of these equations are shown below in eqs. (23) to (30). Using these equations and the range of solutions I found previously I plotted the path each joint followed for that range of angles, shown in Figure 5.

$$\vec{A} = -\vec{R_{CA}} \tag{23}$$

$$\vec{B} = -\vec{R_{CA}} + \vec{R_{BA}} \tag{24}$$

$$\vec{C} = 0 \tag{25}$$

$$\vec{D} = \vec{R_{FC}} + \vec{R_{EF}} + \vec{R_{DE}} \tag{26}$$

$$\vec{E} = \vec{R_{FC}} + \vec{R_{EF}} \tag{27}$$

$$\vec{F} = \vec{R_{FC}} \tag{28}$$

$$\vec{G} = \vec{R_{GC}} \tag{29}$$

$$\vec{P} = -\vec{R_{CA}} + \vec{R_{BA}} + \vec{R_{PB}} \tag{30}$$

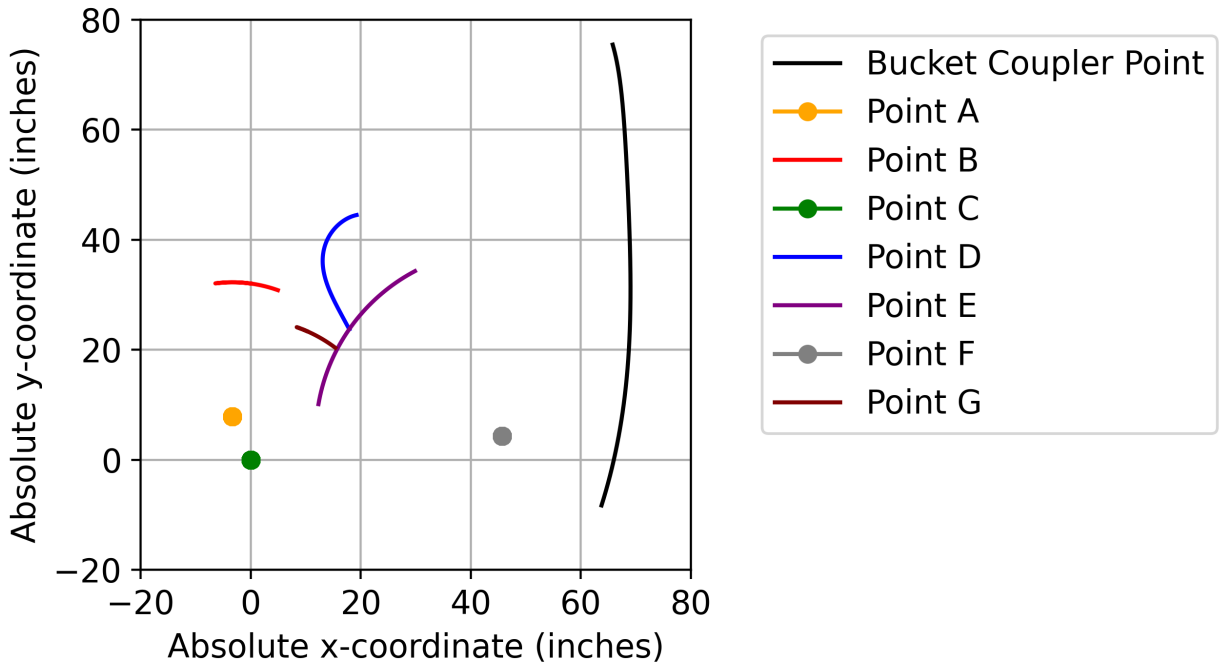

Figure 5: Path of absolute position of linkage joints

Next, I took the list of angles for each input length and used these angles and the previously measured link lengths in eqs. (11) to (22) to solve for angular velocity and angular acceleration of each point. The angular velocity and acceleration of Link BA is plotted in Figure 6 and Figure 7.

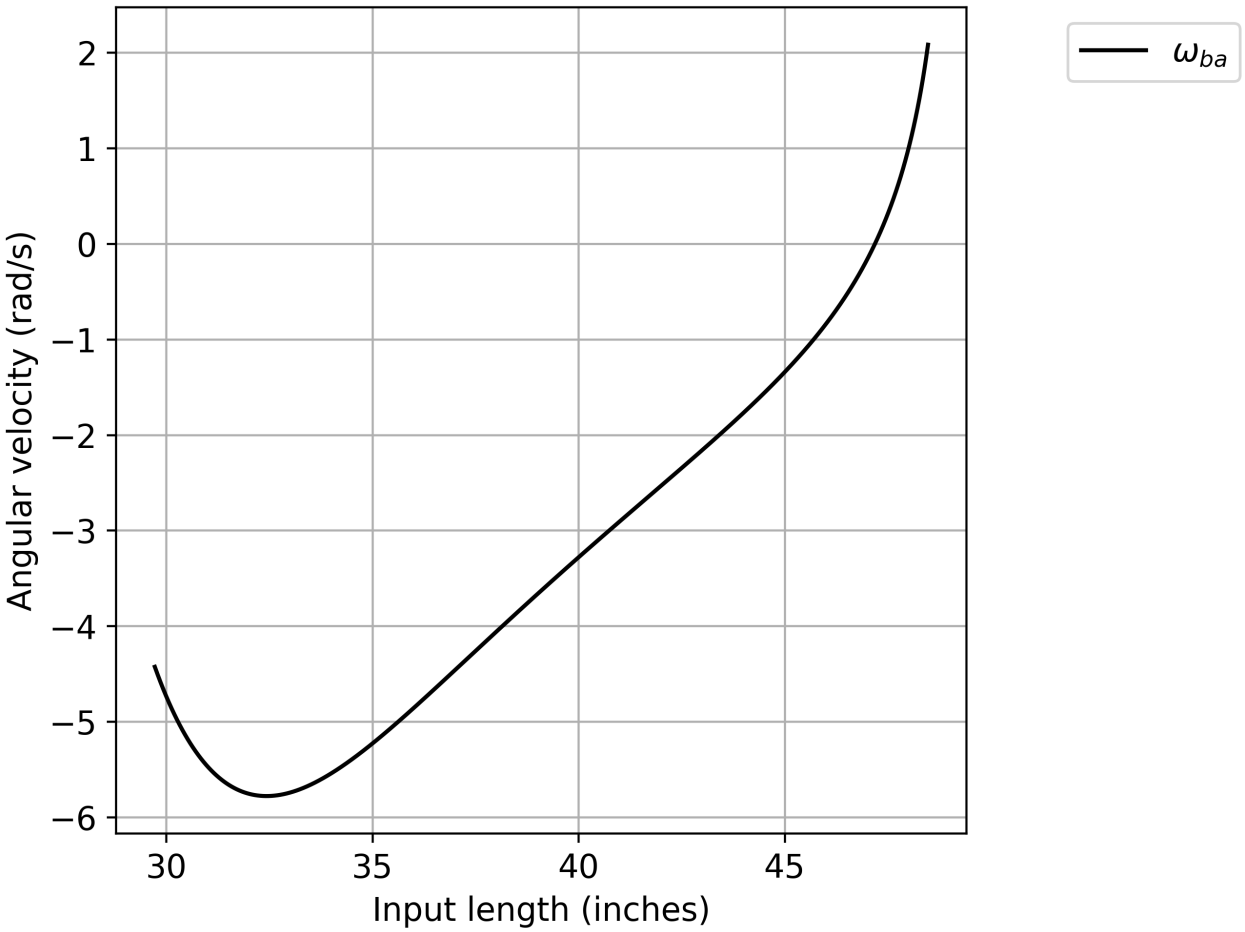

Figure 6: Angular velocity of Link BA vs the input length

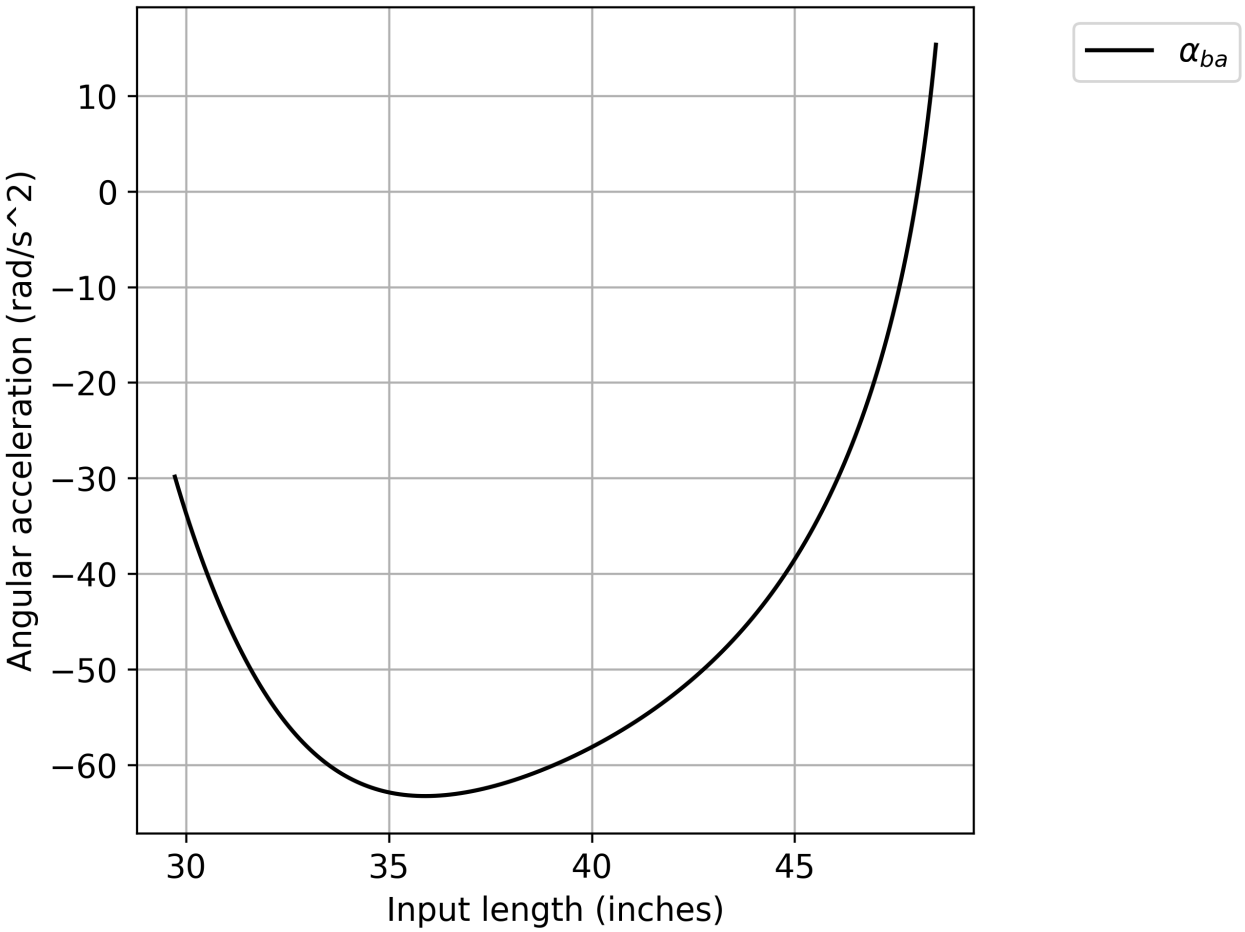

Figure 7: Angular acceleration of Link BA vs the input length

Link Measurements

$$
\begin{aligned}
R_{BA} &= 24.4 \text{ in.} \\
R_{DB} &= 19.8 \text{ in.} \\
R_{CA} &= 8.5 \text{ in.} \\
\theta_{CA} &= 293.04^\circ \\
R_{GC} &= 25.5 \text{ in.} \\
R_{DE} &= 14.7 \text{ in.} \\
R_{EF} &= 33.9 \text{ in.} \\
R_{FC} &= 45.9 \text{ in.} \\
\theta_{FC} &= 5.44^\circ \\
\beta_D &= 92.47^\circ
\end{aligned}
$$

Input Link Length

$$R_{DC} = 29.72 \text{ in.}$$

Multiloop Solver Code

```
import os
import numpy as np
import math
from scipy.optimize import fsolve
import matplotlib.pyplot as plt
from warnings import catch_warnings

act_img_ratio = 0.1718 # Approximate ratio of length without bucket from specs in inches over length without bucket from image in mm

#act_img_ratio = 1     # Uncomment to use image link lengths instead of approximated actual lengths
# Known values (For the moment just measured in millimeters from image multiplied by ratio)
rba = 142.27*act_img_ratio
rdb = 115.2*act_img_ratio
rca = 49.46*act_img_ratio
thetaca = math.radians(293.04)
rgc = 148.5*act_img_ratio # Can't really tell from picture, but should not affect results as long as counted for accordingly
rde = 85.65*act_img_ratio
ref = 197.12*act_img_ratio
rfc = 267.12*act_img_ratio
thetafc = math.radians(5.44) # angle from positive x-axis to ground link, which is Rea
betad = math.radians(92.47) # Angle from DB to DA
rpb = 439.41*act_img_ratio # distance from point b to the bucket coupler point
```

```
23  betab = math.radians(7.41) # Angle from rpb to rdb
24
25
26  # Inputs
27  rdc = 173*act_img_ratio
28  rdotdg = 100 # rate of change of length of hydraulic piston
29  rddotdg = 0 # rate of change of rate of change of length of hydraulic piston
30
31  rdg = rdc - rgc # Given input length
32
33  # Finds most common element in a list
34  def most_common(lst: list):
35      return max(set(lst), key=lst.count)
36
37
38  """
39  Unknowns:
40  x[0]: thetaba
41  x[1]: thetadb
42  x[2]: thetagc
43  x[3]: thetadg
44  x[4]: thetade
45  x[5]: thetaef
46  """
47  unknown_length_indices = []
48  used_inputs = []
49  num_equations = 6
50
51  # System of scalar equations to solve
52  def angle_eqs(x):
53      return [
54          rba*np.cos(x[0]) + rdb*np.cos(x[1]) - rca*np.cos(thetaca) -
        ↪  rgc*np.cos(x[2]) - rdg*np.cos(x[3]),
55          rba*np.sin(x[0]) + rdb*np.sin(x[1]) - rca*np.sin(thetaca) -
        ↪  rgc*np.sin(x[2]) - rdg*np.sin(x[3]),
56          rgc*np.cos(x[2]) + rdg*np.cos(x[3])  -rde*np.cos(x[4]) -
        ↪  ref*np.cos(x[5]) - rfc*np.cos(thetafc),
57          rgc*np.sin(x[2]) + rdg*np.sin(x[3]) - rde*np.sin(x[4]) -
        ↪  ref*np.sin(x[5]) - rfc*np.sin(thetafc),
58          x[2]-x[3],
59          x[1]+betad-x[4]
60      ]
61
62  def angular_velocity_eqs(thetaba, thetadb, thetagc, rdotdg, thetadg, thetade,
    ↪  thetaef, rdc):
63      rdg = rdc - rgc
64      return [
65          [-rba*np.sin(thetaba), -rdb*np.sin(thetadb), rgc*np.sin(thetagc),
        ↪  rdg*np.sin(thetadg), 0, 0],
66          [rba*np.cos(thetaba), rdb*np.cos(thetadb), -rgc*np.cos(thetagc),
        ↪  -rdg*np.cos(thetadg), 0, 0],
67          [0, 0, -rgc*np.sin(thetagc), -rdg*np.sin(thetadg), rde*np.sin(thetade),
        ↪  ref*np.sin(thetaef)],
68          [0, 0, rgc*np.cos(thetagc), rdg*np.cos(thetadg), -rde*np.cos(thetade),
        ↪  -ref*np.cos(thetaef)],
```

```
69          [0, 0, 1, -1, 0, 0],
70          [0, 1, 0, 0, -1, 0]
71      ], [
72          rdotdg*np.cos(thetadg),
73          -rdotdg*np.sin(thetadg),
74          -rdotdg*np.cos(thetadg),
75          -rdotdg*np.sin(thetadg),
76          0,
77          0
78      ]
79
80  def angular_acceleration_eqs(thetaba, thetadb, thetagc, rdotdg, thetadg,
    ↪  thetade, thetaef, rdc, rddotdg, omegaba, omegadb, omegagc, omegadg, omegade,
    ↪  omegaef):
81      rdg = rdc - rgc
82      return [
83          [-rba*np.sin(thetaba), -rdb*np.sin(thetadb), rgc*np.sin(thetagc),
            ↪  rdg*np.sin(thetadg), 0, 0],
84          [rba*np.cos(thetaba), -rdb*np.cos(thetadb), -rgc*np.cos(thetagc),
            ↪  -rdg*np.cos(thetadg), 0, 0],
85          [0, 0, -rgc*np.sin(thetagc), -rdg*np.sin(thetadg), rde*np.sin(thetade),
            ↪  ref*np.sin(thetaef)],
86          [0, 0, rgc*np.cos(thetagc), rdg*np.cos(thetadg), -rde*np.cos(thetade),
            ↪  -ref*np.cos(thetaef)],
87          [0, 0, 1, -1, 0, 0],
88          [0, 1, 0, 0, -1, 0]
89      ], [
90          rba*(omegaba**2)*np.cos(thetaba) + rdb*(omegadb**2)*np.cos(thetadb) -
            ↪  rgc*(omegagc**2)*np.cos(thetagc) + rddotdg*np.cos(thetadg) -
            ↪  2*rdotdg*omegadg*np.sin(thetadg) - rdg*(omegadg**2)*np.cos(thetadg),
91          rba*(omegaba**2)*np.sin(thetaba) + rdb*(omegadb**2)*np.sin(thetadb) -
            ↪  rgc*(omegagc**2)*np.sin(thetagc) + rddotdg*np.sin(thetadg) +
            ↪  2*rdotdg*omegadg*np.cos(thetadg) - rdg*(omegadg**2)*np.sin(thetadg),
92          rgc*(omegagc**2)*np.cos(thetagc) - rddotdg*np.cos(thetadg) +
            ↪  2*rdotdg*omegadg*np.sin(thetadg) + rdg*(omegadg**2)*np.cos(thetadg)
            ↪  - rde*(omegade**2)*np.cos(thetade) -
            ↪  ref*(omegaef**2)*np.cos(thetaef),
93          rgc*(omegagc**2)*np.sin(thetagc) - rddotdg*np.sin(thetadg) -
            ↪  2*rdotdg*omegadg*np.cos(thetadg) + rdg*(omegadg**2)*np.sin(thetadg)
            ↪  - rde*(omegade**2)*np.sin(thetade) -
            ↪  ref*(omegaef**2)*np.sin(thetaef),
94          0,
95          0
96      ]
97
98  # Rename pi for shorter call
99  pi = math.pi
100
101 # For finding right guesses
102 initial_guess_min = [0, 0, 0, 0, 0, 0]
103 initial_guess_max = [2*pi, 2*pi, 2*pi, 2*pi, 2*pi, 2*pi]
104 current_guess = [0, 0, 0, 0, 0, 0]
105
106 # Create empty set to store unique solutions
107 gen_solutions = set()
```

```python

# Check if solution is in approximate range of motion
def result_check(result) -> bool:
    if result[0] > 3*pi/4 or result[0] < pi/4:

        return False
    if result[1] > pi/2 and result[1] < 3*pi/2:

        return False
    if result[2] > pi/2:

        return False
    if result[3] > pi/2:

        return False
    if result[4] > pi:

        return False
    if result[5] < pi/2 or result[5] > pi:

        return False
    return True

# Check error of solution
"""def calc_err(result, eq=eqs):
    err = eq(result)
    err_sum = 0
    for i in range(len(err)):
        err_sum += float(np.abs(err[i]))
    return err_sum"""

def calc_err(result, eq=angle_eqs, unkn_len_ind=unknown_length_indices):
    err = eq(result)
    err_sum = 0
    for i in range(len(err)):
        if i in unkn_len_ind:
            calc_err_amt = float(np.abs(err[i]))
        else:
            calc_err_amt = float(np.abs(err[i])) % 2*pi
            calc_err_amt = min(calc_err_amt, 2*pi - calc_err_amt)
        err_sum += calc_err_amt
    return err_sum


# Populate the solution set with valid solutions
def solution_gen(current_guess: list[float] | None = None, guess_per_var: int =
    5, index: int = 0, initial_min: list[float] = initial_guess_min,
    initial_max: list[float] = initial_guess_max, output: set[tuple[float, ...]]
    = gen_solutions, eq=angle_eqs, num_var: int = num_equations,
    unkn_len_ind=unknown_length_indices):
    if current_guess is None:
        current_guess = []
    if len(initial_min) ≠ num_var and len(initial_max) ≠ num_var:
        exit("Initial and/or final guess are wrong length")

```

```
159     if index == num_var:
160         with catch_warnings(record=True) as recorded_warnings:
161             sol = fsolve(eq, current_guess)
162             curr_error = calc_err(sol, eq)
163             if result_check(sol) and curr_error < 0.1:
164                 output.add(tuple(sol))
165         return
166 
167 
168     guesses = np.linspace(initial_min[index], initial_max[index], guess_per_var)
169 
170     # Finds solutions with all different combinations of guesses
171     for guess in guesses:
172         solution_gen(current_guess=current_guess+[guess],
            ↪ guess_per_var=guess_per_var, index=index+1, initial_min=initial_min,
            ↪ initial_max=initial_max, output=output, eq=eq, num_var=num_var,
            ↪ unkn_len_ind=unkn_len_ind)
173     return
174 
175 
176 # Absolute position of bucket coupler point with point C as the origin, positive
    ↪ x-axis to the right
177 def link_point_pos(angles):
178 
179     point_c = (0, 0)
180 
181     point_a = (-rca*math.cos(thetaca), -rca*math.sin(thetaca))
182 
183     point_f = (rfc*math.cos(thetafc), rfc*math.sin(thetafc))
184 
185     point_b = (point_a[0] + rba*math.cos(angles[0]), point_a[1] +
        ↪ rba*math.sin(angles[0]))
186 
187     point_e = (point_f[0] + ref*math.cos(angles[5]), point_f[1] +
        ↪ ref*math.sin(angles[5]))
188 
189     point_g = (rgc*math.cos(angles[2]), rgc*math.sin(angles[2]))
190 
191     point_d = (point_e[0] + rde*math.cos(angles[4]), point_e[1] +
        ↪ rde*math.sin(angles[4]))
192 
193     bucket = (point_b[0] + rpb*math.cos(angles[1]-betab), point_b[1] +
        ↪ rpb*math.sin(angles[1]-betab))
194 
195     return point_a, point_b, point_c, point_d, point_e, point_f, point_g, bucket
196 
197 
198 
199 # Generate solutions
200 solution_gen()
201 
202 used_inputs.append(rdc)
203 
204 # Turn set into a list so it is indexable
205 gen_solutions = list(gen_solutions)
```

```
min_error = 1
min_sol = None
for sol in gen_solutions:
    err = calc_err(sol)
    if err < min_error:
        min_error = err
        min_sol = sol


s = min_sol

if s is None:
    exit("No solutions found")

sol_domain = np.linspace(rdc, 1.9*rgc, 10000)
rdc_store = rdc
sol_set = [s]
for x in sol_domain[1:]:
    rdc = x
    rdg = rdc - rgc
    with catch_warnings(record=True) as record_warnings:
        try:
            new_sol = fsolve(angle_eqs, sol_set[-1])
            sol_set.append(new_sol)
            used_inputs.append(rdc)
        except:
            continue




rdc = rdc_store
rdg = rdc - rgc

point_ax = []
point_ay = []
point_bx = []
point_by = []
point_cx = []
point_cy = []
point_dx = []
point_dy = []
point_ex = []
point_ey = []
point_fx = []
point_fy = []
point_gx = []
point_gy = []
bucket_x = []
bucket_y = []

omegaba = []
omegadb = []
```

```
261 omegagc = []
262 omegadg = []
263 omegade = []
264 omegaef = []
265
266 alphaba = []
267 alphadb = []
268 alphagc = []
269 alphadg = []
270 alphade = []
271 alphaef = []
272
273 ang_vel_sol_set = []
274 ang_acc_sol_set = []
275 for i in range(len(sol_set)):
276     point_pos = link_point_pos(sol_set[i])
277     point_ax.append(point_pos[0][0])
278     point_ay.append(point_pos[0][1])
279     point_bx.append(point_pos[1][0])
280     point_by.append(point_pos[1][1])
281     point_cx.append(point_pos[2][0])
282     point_cy.append(point_pos[2][1])
283     point_dx.append(point_pos[3][0])
284     point_dy.append(point_pos[3][1])
285     point_ex.append(point_pos[4][0])
286     point_ey.append(point_pos[4][1])
287     point_fx.append(point_pos[5][0])
288     point_fy.append(point_pos[5][1])
289     point_gx.append(point_pos[6][0])
290     point_gy.append(point_pos[6][1])
291     bucket_x.append(point_pos[7][0])
292     bucket_y.append(point_pos[7][1])
293
294     ang_vel_coeff, ang_vel_const = angular_velocity_eqs(sol_set[i][0],
        ↪  sol_set[i][1], sol_set[i][2], rdotdg, sol_set[i][3], sol_set[i][4],
        ↪  sol_set[i][5], used_inputs[i])
295     ang_vel_sol_set.append(np.linalg.solve(ang_vel_coeff, ang_vel_const))
296
297     ang_acc_coeff, ang_acc_const = angular_acceleration_eqs(sol_set[i][0],
        ↪  sol_set[i][1], sol_set[i][2], rdotdg, sol_set[i][3], sol_set[i][4],
        ↪  sol_set[i][5], used_inputs[i], rddotdg, ang_vel_sol_set[i][1],
        ↪  ang_vel_sol_set[i][1], ang_vel_sol_set[i][2], ang_vel_sol_set[i][3],
        ↪  ang_vel_sol_set[i][4], ang_vel_sol_set[i][5])
298     ang_acc_sol_set.append(np.linalg.solve(ang_acc_coeff, ang_acc_const))
299
300
301 ba_ang_vel = [point[0] for point in ang_vel_sol_set]
302 ba_ang_acc = [point[0] for point in ang_acc_sol_set]
303
304 def r_degrs(num):
305     return num/pi*180 % 360
306
307 s = list(s)
308 for i in range(len(s)):
309     if not i in unknown_length_indices:
```

```
        s[i] = r_degrs(s[i])
print(f"Theta_BA = {round(s[0], 3)} degrees")
print(f"Theta_DB = {round(s[1], 3)} degrees")
print(f"Theta_GC = {round(s[2], 3)} degrees")
print(f"Theta_DG = {round(s[3], 3)} degrees")
print(f"Theta_DE = {round(s[4], 3)} degrees")
print(f"Theta_EF = {round(s[5], 3)} degrees")

plot_path = os.path.join(os.path.split(os.path.split(os.path.abspath(__file__))[0])[0], "imgs")

# Link Points Plot
plt.rcParams['font.size'] = 12
fig1, ax1 = plt.subplots(layout='constrained')
ax1.grid()
ax1.plot(bucket_x, bucket_y, 'black', label = 'Bucket Coupler Point')
ax1.plot(point_ax, point_ay, color='orange', marker='o', label = 'Point A')
ax1.plot(point_bx, point_by, 'red', label = 'Point B')
ax1.plot(point_cx, point_cy, color='green', marker='o', label = 'Point C')
ax1.plot(point_dx, point_dy, 'blue', label = 'Point D')
ax1.plot(point_ex, point_ey, 'purple', label = 'Point E')
ax1.plot(point_fx, point_fy, color='gray', marker='o', label = 'Point F')
ax1.plot(point_gx, point_gy, 'maroon', label = 'Point G')
ax1.set_xlim(-20, 80)
ax1.set_ylim(-20, 80)
ax1.set_aspect('equal', adjustable='box')
ax1.set_xlabel('Absolute x-coordinate (inches)')
ax1.set_ylabel('Absolute y-coordinate (inches)')
ax1.legend(bbox_to_anchor=(1.1, 1), loc='upper left')
plt.savefig(os.path.join(plot_path, "LinkPointCoordinates.png"), dpi=400)


plt.rcParams['font.size'] = 12
fig1, ax1 = plt.subplots(layout='constrained')
ax1.grid()
ax1.plot(used_inputs, ba_ang_vel, color='black', linestyle='solid', label = '$\\omega_{ba}$')
ax1.set_xlabel('Input length (inches)')
ax1.set_ylabel('Angular velocity (rad/s)')
ax1.legend(bbox_to_anchor=(1.1, 1), loc='upper left')
plt.savefig(os.path.join(plot_path, "LinkBAAngVel.png"), dpi=400)


plt.rcParams['font.size'] = 12
fig1, ax1 = plt.subplots(layout='constrained')
ax1.grid()
ax1.plot(used_inputs, ba_ang_acc, color='black', linestyle='solid', label = '$\\alpha_{ba}$')
ax1.set_xlabel('Input length (inches)')
ax1.set_ylabel('Angular acceleration (rad/s^2)')
ax1.legend(bbox_to_anchor=(1.1, 1), loc='upper left')
plt.savefig(os.path.join(plot_path, "LinkBAAngAcc.png"), dpi=400)
```

Multiloop Solver Output

```
PS C:\ ... \Honors> python code/main.py
Theta_BA = 82.249 degrees
Theta_DB = 335.111 degrees
Theta_GC = 52.91 degrees
Theta_DG = 52.91 degrees
Theta_DE = 67.581 degrees
Theta_EF = 170.215 degrees
```